

A Calm, Visual Explanation of `myPow(x, n)`

First: breathe

This code looks scary because it is recursive, but the idea is small:

To compute x^n , keep cutting n in half whenever possible.

You are not bad at coding. Your brain is just overloaded. We will slow it down.

The Code

```
1 class Solution:
2     def myPow(self, x: float, n: int) -> float:
3         if n == 0:
4             return 1
5         if n < 0:
6             return 1 / self.myPow(x, -n)
7         if n % 2 == 1:
8             return x * self.myPow(x, n - 1)
9         return self.myPow(x * x, n // 2)
```

What Problem Is This Solving?

The function calculates:

$$\text{myPow}(x, n) = x^n$$

Examples:

$$\text{myPow}(2, 5) = 2^5 = 32$$

$$\text{myPow}(2, -3) = 2^{-3} = \frac{1}{2^3} = \frac{1}{8}$$

The Main Trick

Instead of multiplying one-by-one like this:

$$2^8 = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2$$

the code uses this shortcut:

$$x^n = (x^2)^{n/2} \quad \text{when } n \text{ is even.}$$

So:

$$2^8 = 4^4 = 16^2 = 256^1 = 256$$

Core intuition

When the exponent is even, square the base and cut the exponent in half.

$$\text{myPow}(x, n) \rightarrow \text{myPow}(x \cdot x, n/2)$$

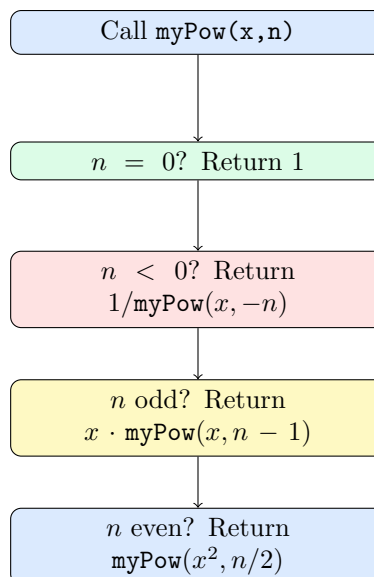
This makes the problem shrink very fast.

Line-by-Line Explanation

Code	Meaning
<code>if n == 0: return 1</code>	Anything to the power 0 is 1. This stops the recursion.
<code>if n < 0: return 1 / myPow(x, -n)</code>	A negative power means reciprocal. Example: $2^{-3} = 1/2^3$.
<code>if n % 2 == 1</code>	If n is odd, remove one x first.
<code>return x * myPow(x, n - 1)</code>	Example: $2^5 = 2 \cdot 2^4$. Now the exponent becomes even.
<code>return myPow(x * x, n // 2)</code>	If n is even, square the base and halve the exponent.

Think of It as Four Doors

Every time the function runs, it asks: “Which door should I go through?”



Example 1: myPow(2, 10)

$\text{myPow}(2, 10) = \text{myPow}(4, 5)$	10 is even: square 2, halve 10
$= 4 \cdot \text{myPow}(4, 4)$	5 is odd: pull out one 4
$= 4 \cdot \text{myPow}(16, 2)$	4 is even
$= 4 \cdot \text{myPow}(256, 1)$	2 is even
$= 4 \cdot 256 \cdot \text{myPow}(256, 0)$	1 is odd
$= 4 \cdot 256 \cdot 1$	
$= 1024$	

So:

$$\text{myPow}(2, 10) = 1024$$

Example 2: myPow(2, -3)

Negative exponent means reciprocal:

$$2^{-3} = \frac{1}{2^3}$$

The code does exactly that:

$$\begin{aligned}\text{myPow}(2, -3) &= \frac{1}{\text{myPow}(2, 3)} \\ &= \frac{1}{2 \cdot \text{myPow}(2, 2)} \\ &= \frac{1}{2 \cdot \text{myPow}(4, 1)} \\ &= \frac{1}{2 \cdot 4 \cdot \text{myPow}(4, 0)} \\ &= \frac{1}{2 \cdot 4 \cdot 1} \\ &= \frac{1}{8}\end{aligned}$$

Why This Is Fast

A simple method might do this for 2^{100} :

$$\underbrace{2 \cdot 2 \cdot 2 \cdots 2}_{100 \text{ times}}$$

This code mostly keeps cutting the exponent in half:

$$100 \rightarrow 50 \rightarrow 25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

That is much fewer steps.

Time complexity

Because the exponent is repeatedly halved, the algorithm runs in:

$$O(\log n)$$

This is called **binary exponentiation** or **fast power**.

The Recursion Idea in Human Words

Recursion means:

Solve a smaller version of the same problem, then use that answer.

For this code:

```
myPow(2, 10)
```

asks:

Can I transform this into an easier power problem?

Then it keeps shrinking the exponent until it reaches:

$$n = 0$$

At that point, it returns 1, and all the previous calls finish.

Mental Model

You can remember the whole code like this:

Memory shortcut

1. If exponent is 0, answer is 1.
2. If exponent is negative, flip it into a reciprocal.
3. If exponent is odd, take out one base: $x^n = x \cdot x^{n-1}$.
4. If exponent is even, square the base and halve the exponent: $x^n = (x^2)^{n/2}$.

Tiny Comfort Note

Feeling stressed does not mean you should leave tech. It means your brain needs smaller chunks, kinder pacing, and more visual explanations. Recursive code often feels confusing at first because it asks you to trust future function calls before you see the result.

You do not need to understand everything instantly. For now, remember just this:

Even exponent: square base, halve exponent. Odd exponent: remove one base.

That is the heart of the algorithm.